

레거시를 끌어올리는 개발자

원재호

Won Jaeho

프론트엔드 중심 풀스택 개발자

레거시 시스템을 모던 스택으로 끌어올리고, 흩어진 코드에 표준을 부여하는 일에 강점이 있습니다. Nexacro 마이그레이션 2회·Vue3 표준화 1회를 거치며, 손이 많이 가는 자리에서 시스템의 결을 잡아왔습니다.

✉ tufc@naver.com

☎ 010-7234-5675

📍 서울특별시 서대문구

생년월일 1997.07.18

경력 2023.03 ~ 현재 (3년차)

소속 주식회사 브렌 · SI사업부

학력 아세아항공직업전문학교 · 항공정비과
(2018.02)

01 핵심 강점



마이그레이션 전문성

Nexacro 14 → N 두 차례, AS-IS → TO-BE(KOPMS) 한 차례. 레거시 코드 분석부터 점진 이관 까지 일관 수행.



시스템 표준화

공용 컴포넌트·매퍼 패턴·SQL 컨벤션 정립. 레거시 시스템의 흩어진 화면 코드를 재사용 가능한 형태로 정리.



풀스택 도메인 적응력

발전·금융·결제·공공·B2B 6개 도메인 경험. Vue3 + Spring Boot 양쪽 모두 자력 구현.



AI 협업 능숙도

Claude Code를 활용한 코드 분석·설계 검토·문서 자동화. 단순 보조가 아닌 메타 분석 수준의 활용.

02 기술 스택

Frontend

Vue 3 능숙

JavaScript 능숙

Vuetify 능숙

Nexacro Platform 능숙

jQuery / JSP 경험

Backend

Spring Boot 능숙

Spring 능숙

MyBatis 능숙

Java 능숙

Database

Oracle 능숙

PostgreSQL 경험

MariaDB 경험

DB2 경험

MS-SQL 경험

Tools & AI

Claude Code 능숙

Git / SVN 능숙

Maven 능숙

STS / Eclipse 능숙

VS Code 능숙

03 경력 타임라인

2023년 입사 이후 6개 프로젝트, 5개 도메인을 거치며 풀스택 역량과 마이그레이션 전문성을 다져왔습니다.

2025.12 ~ 2026.06 (7개월)

KOPMS 신사업관리시스템 고도화 메인

한국남동발전 · 프론트엔드, 백엔드

Vue 3

Vuetify

Spring Boot

MyBatis

Oracle

2025.04 ~ 2025.11 (7개월)

백오피스 시스템(VMS) 고도화

스마트로 · 프론트엔드

Nexacro 14

Nexacro N

Spring

2024.10 ~ 2025.01 (4개월)

Smartro QR Order 서비스 개발

스마트로 · 프론트엔드, 백엔드

Vue 3

Spring Boot

DB2

2024.02 ~ 2024.09 (8개월)

SMART INVOICE 서비스 개발

자체 서비스 · 프론트엔드, 백엔드

Vue 3

Spring

PostgreSQL

2023.08 ~ 2023.11 (4개월)

MRS UI 고도화 프로젝트

IBK 신용정보 · 프론트엔드

Nexacro 14

Nexacro N

2023.05 ~ 2023.07 (3개월)

한국전력 통합 프레임워크 구축

한국전력기술 · 프론트엔드, 백엔드

Nexacro N

Spring

MariaDB

04 메인 프로젝트 KOPMS 신사업관리시스템

고객사 한국남동발전 **기간** 2025.12 ~ 2026.06 (7개월) **역할** 프론트엔드, 백엔드

스택 Vue 3 Vuetify Spring Boot MyBatis Oracle

AS-IS JSP 기반 레거시 시스템을 Vue 3 + Spring Boot로 전면 마이그레이션. 요구 분석·화면 설계부터 UI 구현, 백엔드·DB, QA까지 전 영역을 담당했고, 기존 화면을 그대로 옮기지 않고 정보구조·동선을 다시 설계해 재구성. 공용 컴포넌트·SQL 패턴·백엔드 패키지 구조까지 표준을 정립.

01 화면 전면 재구성 · 공용 컴포넌트 시스템 설계

- **AS-IS JSP 화면 제로베이스 재구성** — 단순 이관이 아니라 정보구조·사용자 동선을 다시 설계해 Vue 3로 구현. 시안 의존 없이 화면 설계부터 구현까지 자력 수행.
- **BrFileAttach / BrFileInput** — 첨부파일 단일·다중 패턴 일원화. proStep+seqColumn 자동 매핑, stale race 방지 패턴 내장.
- **BrVersionSelect** — 이력 버전 셀렉트 공용화. 현재/이력 분기, displayLabel 슬롯 패턴.
- **BrDateField / BrNumberField / EmpSearchField** — 폼 입력 컴포넌트 표준화. prefix/suffix 슬롯 + jdbcType=VARCHAR 적용 가이드.
- **br-table-fill / br-tab / score-panel** — 글로벌 utility class로 디자인 가이드 자동 반영.

02 백엔드 구조 리팩토링

- 같은 특성 서비스끼리 패키지 분리 → 프론트 폴더구조와 백엔드 패키지 일치시켜 유지보수성 향상.
- MyBatis OGNL 분기 → SQL CASE WHEN 우선 정책 도입. 가독성·디버깅 개선.
- BIZ_MASTER / BIZ_PORTAL / NBM_BIZ_MASTER 등 핵심 테이블의 proStep 스코프 격리 + 역할코드 정리.

03 AS-IS → TO-BE 매핑 및 검증

- 매퍼 grep 분석으로 실제 사용 테이블 추출 → DSN 산출물(테이블 목록/정의서) 정합성 검증 자동화.
- 24건 잉여 정의서 식별 + 6건 누락 정의서 추가. 3중 매트릭스(DSN_09 ↔ DSN_10 ↔ 매퍼) 비교 스크립트 작성.
- AS-IS 마이그레이션 정책 준수 — 위험 발견 시 임의 개선 대신 협의 항목으로 분리.

04 전자결재(MIKEP) 연동 / 첨부파일 / 이력 추적

- SIGN_DOC/SIGN_LINE XML 구조 기반 결재 호출 흐름 구현 (TRANSACTION_KEY 패턴, gianCall.do 팝업).
- BIZ_ATTACH_FILE TASK_TYPE/CODE 길이 제약 발견 → 표준 코드 체계 정립.
- BIZ_PORTAL_CHANGE_LOG 기반 변경 이력 시스템 + BrVersionSelect 연동.

주요 성과

- ✓ 공용 컴포넌트 10여 종 정립 → 신규 화면 개발 시간 단축
- ✓ 백엔드 매퍼 분기 SQL 단순화로 디버깅 시간 감소
- ✓ DSN 산출물 매트릭스 검증 자동화로 휴먼 에러 사전 차단

05 다른 프로젝트

발전·금융·결제·B2B 등 다양한 도메인에서 풀스택으로 기여한 5개 프로젝트입니다.

백오피스 시스템(VMS) 고도화

2025.04 ~ 2025.11 (7개월)

스마트로 · 프론트엔드

Nexacro 14

Nexacro N

Spring

레거시 Nexacro 14 화면을 Nexacro N으로 마이그레이션. 결제 정산·가맹점 관리 백오피스 모듈 다수 담당.

- Nexacro 14 → N 마이그레이션 (두 번째 경험으로 전 단계 노하우 적용)
- 레거시 비즈니스 로직 보존 + 모던 UI 컨벤션 적용 균형
- 결제 도메인 데이터 흐름 이해 및 검증 케이스 작성

Smartro QR Order 서비스 개발

2024.10 ~ 2025.01 (4개월)

스마트로 · 프론트엔드, 백엔드

Vue 3

Spring Boot

DB2

QR 코드 기반 매장 주문 서비스 신규 개발. 백엔드 API 설계부터 프론트 화면까지 풀스택 수행.

- Vue 3 + Spring Boot 풀스택 신규 서비스 구축
- DB2 환경에서 주문/결제 도메인 모델링
- QR 스캔 → 주문 → 결제 흐름 end-to-end 설계

SMART INVOICE 서비스 개발

2024.02 ~ 2024.09 (8개월)

자체 서비스 · 프론트엔드, 백엔드

Vue 3

Spring

PostgreSQL

전자세금계산서 SaaS 서비스 신규 개발. PostgreSQL 기반 트랜잭션 설계 및 Vue 3 UI 구현.

- 전자세금계산서 도메인 모델링 및 API 설계
- PostgreSQL 트랜잭션 격리 수준 고려한 정산 로직
- Vue 3 기반 입력 폼 + 그리드 UI 일관성 확보

MRS UI 고도화 프로젝트

2023.08 ~ 2023.11 (4개월)

IBK 신용정보 · 프론트엔드

Nexacro 14

Nexacro N

신용정보 MRS 화면을 Nexacro 14에서 N으로 마이그레이션. 첫 마이그레이션 프로젝트로 패턴 학습.

- Nexacro 14 → N 마이그레이션 첫 경험 — 차이점 학습
- 금융권 보안 요구사항 준수하며 UI 이관
- 화면 단위 책임 분담 + QA 반영 흐름 체득

한국전력 통합 프레임워크 구축

2023.05 ~ 2023.07 (3개월)

한국전력기술 · 프론트엔드, 백엔드

Nexacro N

Spring

MariaDB

한전 그룹사 공통 프레임워크 구축 프로젝트. 입사 직후 첫 투입으로 기본기 형성.

- Spring 기반 백엔드 + Nexacro N 프론트 풀스택 첫 경험
- 대형 SI 환경의 표준 가이드/산출물 체계 학습
- Java/JS/Oracle 실무 토대 확립

06 AI 활용 역량

AI를 단순 코드 생성 도구가 아닌, 코드베이스를 함께 분석하고 설계를 검토하는 페어 프로그래밍 파트너로 활용합니다. KOPMS 프로젝트에서 Claude Code를 적극 도입하여 실제 생산성을 끌어올렸습니다.

활용 철학

도구가 아닌 협업자로 본다

단순히 "코드 짜줘"가 아니라, 함께 코드베이스를 읽고 패턴을 발견하고 의사결정을 검토하는 동료처럼 활용합니다.

최종 판단은 사람이 한다

AI 제안을 그대로 받아들이지 않습니다. 메모리/룰 기반으로 프로젝트 컨벤션을 명시하고, AS-IS 마이그레이션 정책 등은 임의 개선 금지로 가드립니다.

맥락 관리에 신경 쓴다

프로젝트 메모리 파일을 적극 활용해 컴포넌트 사용법·DB 스키마·코드 컨벤션을 영속화하여 매 세션마다 일관된 협업 환경을 유지합니다.

실전 활용 사례 — KOPMS 프로젝트

CASE 01 DSN 산출물 정합성 자동 검증

문제 AS-IS 산출물(테이블 목록·정의서)이 실제 사용 테이블과 일치하는지 수기 검증 부담.

접근 MyBatis 매퍼 XML을 grep으로 파싱 → 실제 사용 테이블 추출 → DSN_09 / DSN_10 / 매퍼 3종 매트릭스 비교 스크립트 작성.

결과 잉여 정의서 24건·누락 정의서 6건 자동 식별. 휴먼 에러 차단 + 산출물 신뢰도 향상.

CASE 02 공용 컴포넌트 패턴 합의 및 표준화

문제 신규 화면 추가 시 다이얼로그·테이블·점수판 등이 화면별로 제각각.

접근 기존 화면 코드 분석으로 공통 패턴 추출 → BrFileAttach·br-table-fill·score-panel 등 글로벌 utility로 추출. 메모리에 사용 가이드 영속화.

결과 신규 화면 개발 시 컴포넌트 import만으로 디자인 가이드 자동 반영. 코드 재사용성 향상.

CASE 03 백엔드 패턴 일관성 가드

- 문제** MyBatis OGNL 분기-jdbcType 누락·string compare 등 미세한 컨벤션 차이로 인한 버그 발생.
- 접근** 발견한 컨벤션을 메모리에 명시 — "OGNL 큰따옴표 패턴", "BrNumberField → jdbcType=VARCHAR", "CASE WHEN 우선" 등.
- 결과** 같은 패턴 실수 재발 방지. 코드 리뷰 시간 단축. 신규 화면도 동일 컨벤션 자동 준수.

CASE 04 AS-IS 마이그레이션 정책 자동 가드

- 문제** AS-IS 코드 마이그레이션 시 위험 발견하면 본능적으로 개선하고 싶은 충동 → 고객사 정책 위반.
- 접근** "AS-IS 마이그레이션 코드 임의 개선 금지" 룰을 메모리에 명시. 위험 발견 시 자동으로 협의 항목으로 분리 제안.
- 결과** 고객사 정책 100% 준수 + 위험 항목 별도 트래킹으로 운영 안정성 확보.

이러한 활용 방식은 단순 코드 생성 보조를 넘어, 시스템 전반의 일관성을 유지하는 메타 분석 도구로 AI를 활용하는 방법입니다. 앞으로도 AI 도구의 발전을 적극 학습하며, 더 큰 시스템에서 더 일관된 결과물을 만들어내는 개발자가 되고자 합니다.

07 마무리

가치관

맥락 위에서 결정한다

문제를 발견하면 즉시 고치고 싶은 충동이 들지만, 고객사 정책·운영 영향·기존 컨벤션 등 맥락을 먼저 확인합니다. "왜 이렇게 되어 있는지"를 이해한 후에 손을 겁니다.

재사용 가능한 형태로 정리한다

한 번 풀린 문제는 다음에 같은 문제가 와도 풀릴 수 있도록 패턴화합니다. 공용 컴포넌트·utility class·메모리 풀 등은 모두 이 원칙의 산물입니다.

책임지는 코드를 쓴다

내가 작성한 코드가 운영에서 어떻게 동작할지 끝까지 추적합니다. 단순 화면 구현이 아닌, 데이터가 어떻게 흘러 어디에 저장되는지 이해하고 코드를 작성합니다.

앞으로 학습 방향

- 인프라/배포 영역 학습 — Linux 명령·로그 분석·CI/CD 파이프라인
- Spring Security / SSO(SAML, OAuth 2.0, OIDC) 인증 구조 심화
- Docker / Kubernetes 기반 컨테이너 배포 경험
- AI 활용 사례 지속 발굴 및 팀 내 공유

레거시를 끌어올리고, 흩어진 것을 정리하며, 더 큰 시스템에서도 일관된 결을 만들어내는 개발자가 되고 싶습니다. 읽어주셔서 감사합니다.

원재호

tufc@naver.com · 010-7234-5675